

Accounting App — Roadmap

Engineering · updated 16 Aug 2026

August 16, 2026

CONTENTS

Roadmap	
Phase 0 — Stop the bleeding (1 week)	
Phase 1 — Make the server the system of record (4–6 weeks)	
Phase 2 — Trustworthy books (3–4 weeks)	
Phase 3 — Product hardening (3–4 weeks)	
Phase 4 — Indian compliance depth (4–6 weeks)	
Phase 5 — Scale and reach (ongoing)	
Sequencing notes	

Roadmap

Companion to [REVIEW.md](#). Gap IDs (A-1, C-2, ...) refer to that document. Sizing assumes one full-time developer; run phases in order — each one is a prerequisite for the next.

Progress as of 16 Aug 2026: Phase 0 is complete and Phase 1 has started.

Checked boxes below have shipped on the `phase-0-hardening` branch, with the commit and its evidence recorded in [STATUS.md](#).

Phase 0 — Stop the bleeding (1 week)

Cheap, high-leverage, no architecture change.

- ☒ `git init`, first commit, branch protection. Nothing below is safe without it. (B-7)
- ☒ Add `.env`, `*.db`, `dist/`, `server/dist/`, `logs/`, `tmp/`, `_backup_before_onedrive_restore/` to `.gitignore`; **rotate** `JWT_SECRET`. (B-3)
- ☒ Delete the hardcoded owner email in `src/App.jsx:9068`. (C-1)
- ☒ Add `requirePermission('SALES', ..., 'Invoices')` to all five invoice routes. (C-2)
- ☒ Scope login lookup by account, or make email globally unique. (C-3)
- ☒ Fix the server build (tsconfig → NodeNext + `.js` specifiers) so `npm run build && npm start` boots. (B-1)
- ☒ Replace the PowerShell VS Code tasks with cross-platform npm scripts. (B-5)
- ☐ `docker compose up` green end to end — **still unverified**: the Docker daemon on the dev machine stopped responding mid-build. (B-2)
- ☒ Delete the dead Sequelize backend. (D-4)

Exit: repo versioned, no known privilege bypass, one command starts the stack.

Status: **met**, except the Docker end-to-end run.

Phase 1 — Make the server the system of record (4–6 weeks)

The core rewrite. Everything else depends on it.

- ☐ Postgres + a real `prisma/migrations` history; `Decimal(18,4)` on every money column. (D-1, A-7)
- ☐ Model the domain server-side: `Customer`, `Vendor`, `Item`, `Account` (chart of accounts), `Voucher` + `VoucherLine`, `Payment`, `Expense`, `Estimate`, `Bill`, `CreditNote`, `DebitNote`, `PurchaseOrder`. (D-2)
- ☒ **General ledger**: immutable `JournalEntry` + `JournalLine`. Every voucher posts inside one transaction; unbalanced entries are rejected; control accounts referenced by ID. Reversal by contra entry, never by mutation. (A-3)
- ☐ Server-side voucher numbering with a per-(org, branch, series) sequence. (A-8)
- ☐ CRUD + list APIs with pagination, filtering, and consistent `accountId + orgId + branchId` scoping. (D-6)
- ☐ Frontend switches from `localStorage`-as-database to API + a query cache (TanStack Query). Keep `localStorage` strictly as an offline read cache. (A-1, A-2)
- ☐ One-time importer: read an existing browser `accountingDB:*` blob and push it to the server, so current prototype data survives the migration.
- ☒ Tenancy isolation test suite (org A must never see org B) + ledger math tests — 11 passing. (B-4)

Exit: clearing browser storage loses nothing; two devices show the same book; trial balance is derived from posted journal lines and foots to zero.

Phase 2 — Trustworthy books (3–4 weeks)

- ☒ Fiscal years + period close — a `books locked through <date>` marker; posting into a locked period returns 409. Shipped early alongside the ledger. A separate reopen permission is still to do. (A-4)
- ☐ Audit log on every financial mutation: who, when, before/after. Posted entries are already immutable and hash-chained; the per-field log is not built. (A-5)
- ☐ Year-end closing entries and opening-balance carry-forward.

- ☐ Reports rewritten against the GL: Trial Balance, P&L, Balance Sheet, Cash Flow, Ledger, Day Book, Aged Receivables/Payables.
- ☐ Reconciliation rules: invoice $\rightleftharpoons$ payment allocation, credit-note application, partial payments, write-offs.
- ☐ Negative-stock guard on transfers; apply `InventoryAdjustment` to `StockBalance`; one item catalog shared by UI and server. (D-3, D-5)

Exit: an accountant can close a month and the numbers don't move afterwards.

Phase 3 — Product hardening (3–4 weeks)

- ☐ Split `App.jsx` (10.5k lines) into route-level modules; adopt `react-router`; lazy-load reports. (E-1, E-2)
- ☐ Permission set delivered by `/auth/me`; a `<Can>` gate so UI and API share one vocabulary. (E-3, C-4 — *move bootstrap out of the permission check*)
- ☐ Replace `alert()` / `confirm()` with toasts + dialogs; global 401/403 handling. (E-4)
- ☐ Auth hardening: rate limiting, account lockout, `crypto.randomBytes` reset tokens in the DB, refresh-token rotation, real logout, email verification. (C-5, C-6, C-7)
- ☐ Validate `orgIds` / `branchIds` on user creation against the caller's account. (C-8)
- ☐ Replace `$queryRawUnsafe` in `invoices.ts` with the typed Prisma client. (C-9)
- ☐ Structured logging (pino) + request IDs, Sentry, `/metrics`, nightly backups with a documented restore drill.
- ☐ CI: lint + typecheck + tests + docker build on every PR.

Exit: an on-call engineer can diagnose a production issue from logs alone.

Phase 4 — Indian compliance depth (4–6 weeks)

- ☐ GSTR-1 and GSTR-3B export in the government JSON schema (today they are on-screen reports only).

- ☐ e-Invoice: IRN generation + signed QR via an IRP; QR on the printed invoice.
- ☐ e-Way bill generation and cancellation.
- ☐ GSTR-2B download and purchase reconciliation (matched / mismatched / missing).
- ☐ TDS and TCS: sections, rates, thresholds, challan tracking, Form 26Q data.
- ☐ Reverse charge, export/SEZ invoices with LUT, composition scheme handling.
- ☐ HSN summary and document summary as required by GSTR-1 tables.

Exit: a filing can be produced from the app without a spreadsheet in between.

Phase 5 — Scale and reach (ongoing)

- ☐ Bank statement import (CSV/OFX/MT940) + reconciliation UI — the `bankTransactions` array already exists and nothing populates it.
 - ☐ Recurring invoices, dunning/payment reminders, customer portal, payment links.
 - ☐ Multi-currency with FX revaluation; cost centres, budgets, project accounting.
 - ☐ Background job queue for PDF/export/email; object storage for attachments.
 - ☐ Virtualized data grids, saved views, bulk actions, global search. (E-6)
 - ☐ Mobile-responsive pass and accessibility audit. (E-5)
 - ☐ Read-only auditor role; org-to-org data export (Tally/Zoho/QuickBooks shapes).
-

Sequencing notes

- **Do not** start Phase 4 before Phase 1. Compliance exports built on a derived-at-render-time ledger will have to be rewritten.
- Phase 3's `App.jsx` split is easiest *during* Phase 1, while call sites are already being touched to swap `localStorage` for API calls — consider merging them.
- Each phase should end with a tagged release and a restore-from-backup drill.